AI-Driven Smart Parking Management

Saravanan V
Dept. of Computer Science
Vels Institute of Science, Technology & Advanced Studies (VISTAS)
Chennai, India

Ragul S
Dept. of Computer Science
Vels Institute of Science, Technology & Advanced Studies (VISTAS)
Chennai, India

Maharaja K
Dept. of Computer Science
Vels Institute of Science, Technology & Advanced Studies (VISTAS)
Chennai, India

Ms. Hariprabha
Assistant Professor,
Dept. of Computer Science
Vels Institute of Science, Technology & Advanced Studies (VISTAS)
Chennai, India

***Abstract*—Even in multi-level parking garages, when a car arrives without a reservation or parked incorrectly after entering the parking area, the parking process remains inefficient. This paper proposes an AI-based parking-management system that combines vehicle-detection technology based on YOLO11n, automatic-number-plate-recognition (ANPR) technology based on EasyOCR, booking-verification technology, walk-in-vehicle-management technology and a dynamic slot-allocating technology as an integrated parking-session workflow. An overhead camera is employed to check assigned parking areas for slot mismatch, improper and double parking, and obstruction of driving aisles. A Random Forest model predicts parking availability 30, 60, and 90 minutes in advance, and a rule-based, configurable billing module computes parking fees according to different rules and parking durations. The backend is developed using FastAPI + SQLite and it includes two distinct portals: Admin and User for parking management, booking, monitoring, billing and notifications. A Software Prototype shows the flow of booked and walk-in vehicles from entry to slot release. The anomaly detection and billing modules are described and incorporated in the architecture, and quantitative evaluation on real-world labelled data is left for future work Physical sensing and barrier hardware are planned for a second phase.**

***Index Terms*—Smart parking, artificial intelligence, ANPR, YOLO11n, EasyOCR, walk-in parking, dynamic slot allocation, parking anomaly detection, dynamic billing, Random Forest, FastAPI, SQLite.**

I. Introduction

The number of cars on the road is increasing and urban parking services are expanding at an even faster rate. More parking space can be provided. In malls, offices and hospitals, drivers are often known to idle for long periods of time in front of campuses and airports. When a free parking space is available, this results in fuel being wasted, and it has a negative impact both on traffic and on all the relevant parties concerned [1]. Nevertheless, there is still a place for parking attendants, paper tickets and the like in various facilities even in the face of all these advances. The number of passengers is generally not accurately known and has to be estimated by visual inspection, and errors and breaches in the bills go undetected.

These problems are addressed by smart parking systems through sensing and automation [1]. Camera based designs [2], [3] use computer vision to monitor many slots with little hardware per slot. Automatic Number Plate Recognition (ANPR) is used to identify vehicles at the gate and allow ticketless operation [4], [5]. Reservation platforms allow customers to reserve a slot prior to arrival [6], and machine learning algorithms forecast availability to enable drivers and operators to plan [7], [8].

Most existing solutions, however, treat these functions independently. A booking system does not have a solution for a vehicle arriving without a reservation, while an occupancy detector does not determine whether a vehicle is parked in its assigned slot, occupying multiple spaces, or obstructing a driving aisle . Walk-in vehicles are common and the absence of a booking should not automatically classify a vehicle as unauthorized. By monitoring a vehicle from entry through its parking session and exit, an opportunity arises to combine reservation verification, dynamic allocation, parking compliance monitoring and billing in one workflow.

We propose a system that follows the complete parking session. Its objectives are to detect vehicles, read number plates, verify reservations, admit walk-in vehicles, allocate slots dynamically, predict parking availability, detect parking anomalies, calculate parking charges, and provide user notifications. The main contributions are:

- an integrated parking-session workflow for reserved and unbooked vehicles using YOLO11n-based vehicle detection and EasyOCR-based ANPR;
- dynamic slot allocation that considers only slots that are available and unreserved;
- ROI-based detection of slot mismatch, improper or double parking, and driving-aisle obstruction using explicitly defined decision rules;
- rule-based dynamic billing combined with Random Forest availability prediction; and
- a FastAPI and SQLite implementation with separate Admin and User portals.

The rest of the paper is organized as follows. Section II reviews related work, Section III describes the system and Section IV its methods. Section V covers the implementation, Section VI the experimental setup and results, Section VII the limitations and future work, and Section VIII concludes.

II. Related Work

A. Vision-Based Occupancy Detection

Camera-based occupancy detection replaces per-slot sensors with image analysis of a whole parking area. Almeida et al. released PKLot, a labelled set of parking-lot images taken under different weather conditions that is widely used to benchmark occupancy classifiers [2]. Amato et al. ran a compact convolutional network on smart cameras to classify slot occupancy in a decentralized way [3]. Detectors from the

YOLO family [9], [10] locate several vehicles in one forward pass, which suits overhead views of a parking area; they report where vehicles are, though, not whether a vehicle is where it should be.

B. License Plate Recognition

ANPR pipelines usually locate the plate, clean up the cropped region and then run optical character recognition. Laroca et al. built a real-time system around the YOLO detector [4], and Silva and Jung addressed plates seen under strong perspective distortion in unconstrained scenes [5]. Open-source OCR engines such as EasyOCR [11] remove much of the manual character segmentation, although accuracy still depends on plate visibility, resolution and viewing angle. Simple post-processing, such as stripping spaces and fixing commonly confused characters, makes the recognized string more consistent.

C. Slot Management and Reservation

Henaien et al. [12] proposed a TinyML-enabled system that dynamically computes free slots and recognizes license plates on resource-constrained devices. It is the nearest reference to our work on slot computation combined with plate recognition. Geng and Cassandras proposed a smart parking system based on resource allocation and reservations [6]. Lin et al. survey the sensing, communication and service components of smart parking solutions [1]. The focus of these works is on sensing, allocation and reservation, and not primarily on monitoring compliance during the stay and billing. What happens after entry is where our system differs. Our system checks reservations, allows vehicles without bookings, monitors compliance and calculates the charge.

D. Availability Prediction

Availability prediction is often formulated as a regression or time-series problem over historical occupancy. Zheng et al. compared several machine learning regressors to predict car-park availability from sensor data [7], and Vlahogianni et al. proposed a real-time parking prediction system for smart cities [8]. Ensemble methods such as Random Forest [13] are good for small tabular datasets as they can model non-linear relationships, require little tuning and resist overfitting. The quality of prediction still depends on the amount of history available and its realism.

E. Research Gap

The main focus of the studies reviewed is summarized in Table I. The current work generally deals with occupancy detection, license plate recognition, reservation or availability prediction as separate components. Some systems combine two or more of these functions, but integrated handling of reserved and walk-in vehicles, dynamic slot allocation, in-stay parking compliance, anomaly detection, and session-based billing are not commonly addressed within a single software workflow. The proposed system unites these functions through a common parking-session architecture.

TABLE I
PRIMARY FOCUS OF RELATED STUDIES

Study	Occ.	ANPR	Resv.	Pred.	Anom.	Bill.
Almeida et al. [2]	✓	–	–	–	–	–
Amato et al. [3]	✓	–	–	–	–	–
Laroca et al. [4]	–	✓	–	–	–	–
Silva and Jung [5]	–	✓	–	–	–	–
Geng and Cassandras [6]	–	–	✓	–	–	–
Zheng et al. [7]	–	–	–	✓	–	–
Vlahogianni et al. [8]	–	–	–	✓	–	–
Henaien et al. [12]	✓	✓	–	–	–	–
Proposed system	✓	✓	✓	✓	✓	✓

Occ. = occupancy detection; Resv. = reservation; Pred. = availability prediction; Anom. = in-stay anomaly detection; Bill. = dynamic billing. A check mark indicates the primary focus of a study, not its capabilities.

III. PROPOSED SYSTEM

Fig. 1 depicts the general architecture and Fig. 2 the parking-session workflow. The vehicle is captured by a camera at the entry, detected by YOLO11n, and the number plate is read from the extracted region using EasyOCR. The recognized plate is sent to the FastAPI backend and checked for an active booking, then the booked or walk-in session is opened. A free slot is dynamically allocated and the parking session is started. When the vehicle is parked, an overhead camera checks the parking area for slot mismatch, improper or double parking and driving-aisle obstruction. At the exit the vehicle plate is read again, the appropriate parking charge is computed and the slot is released.

It has five layers of architecture. The perception layer consists of the cameras for the entry, exit, and parking area, which deliver frames for vehicle detection and plate recognition. The AI processing layer uses YOLO11n for vehicle detection and EasyOCR to extract and normalize the plate text. The backend layer, implemented with FastAPI, performs reservation checks, walk-in vehicle processing, slot allocation, parking session management, anomaly detection, availability prediction, billing, and notification handling. The database layer is based on SQLite, where users, vehicles, parking slots, bookings, parking sessions, anomalies, payments and notifications are stored. The application layer provides is providing different portals for Admin and User for monitoring, booking, parking help, billing and notifications.

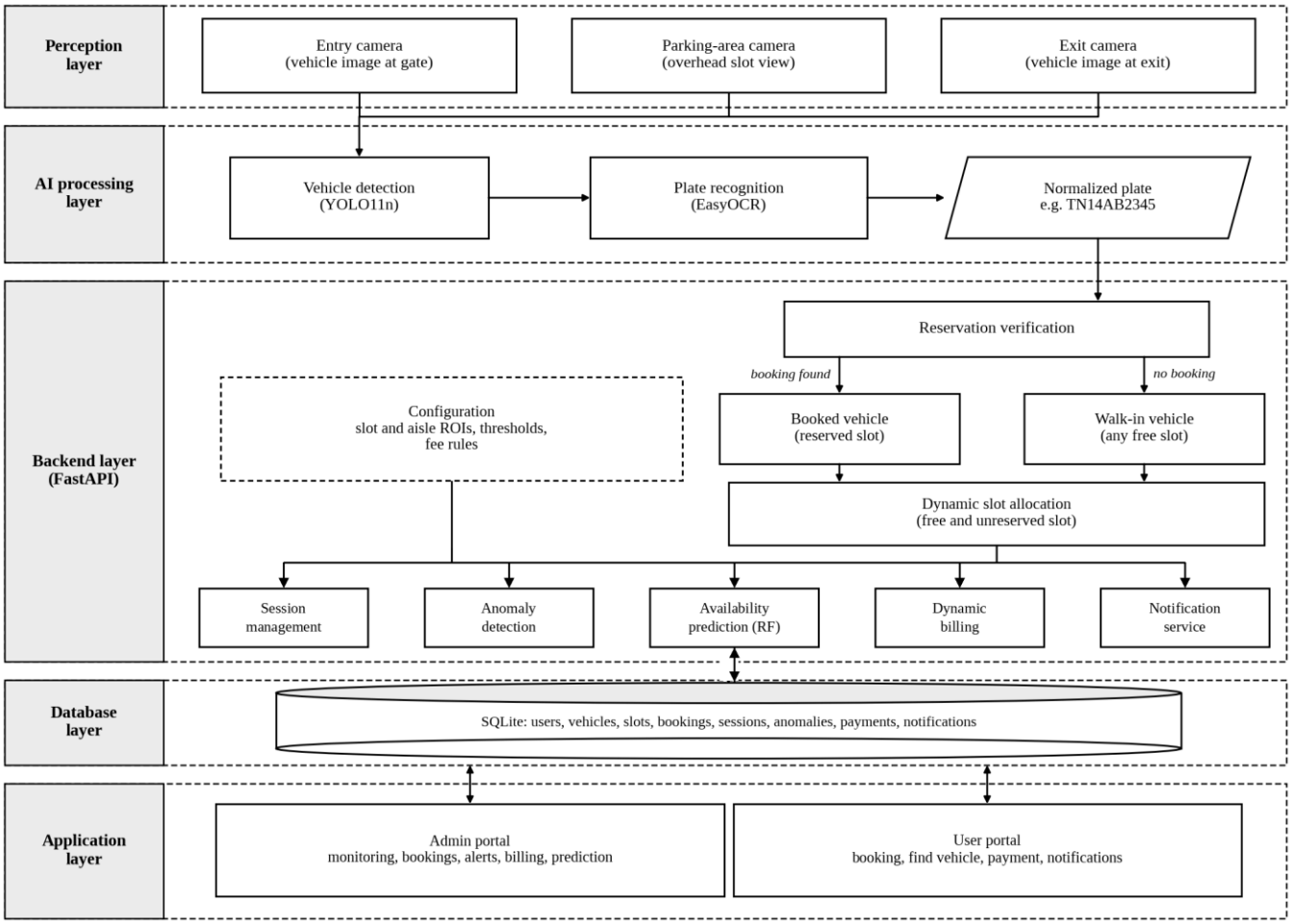


Fig. 1. Overall system architecture of the proposed parking management system.

IV. Methodology

A. Vehicle Detection

Each captured frame is resized and normalized before being fed to the YOLO11n detector [9], [10]. The network predicts bounding boxes and class labels, in one forward pass. We retain any detections that are classified as vehicles and discard any detections below a configurable confidence threshold. Chain is: frame -> preprocessing -> YOLO inference -> vehicle box and class.

B. Automatic Number Plate Recognition

For every detected vehicle, the plate region is located and cropped. The crop is pre-processed, e.g. by grayscaling and contrast enhancement and fed to EasyOCR [11]. The raw output is normalized by removing spaces and punctuation and converting to upper case so that it can be matched with database records; "TN 14 AB 2345" becomes TN14AB2345. These steps are described in Fig. 3.

C. Reservation Verification and Walk-in Handling

The normalized plate is searched in the database. If there is an active booking, the vehicle is considered booked and its slot is utilized. Otherwise it goes into the walk-in process. A missing booking does not make a vehicle unauthorized. In case a parking slot is found to be available and unreserved, the system generates a parking session of type WALK_IN. The driver is told that parking is full only when there are no suitable slots (Fig. 2).

D. Dynamic Slot Allocation

Two conditions must be met by the candidate slots: the slot is free and the slot is not booked. The remaining slots are then filtered according to configurable criteria like proximity to entrance, preferred floor, preferred section etc. After allocation, the slot status is updated, so the same slot is not offered to another vehicle at the same time.

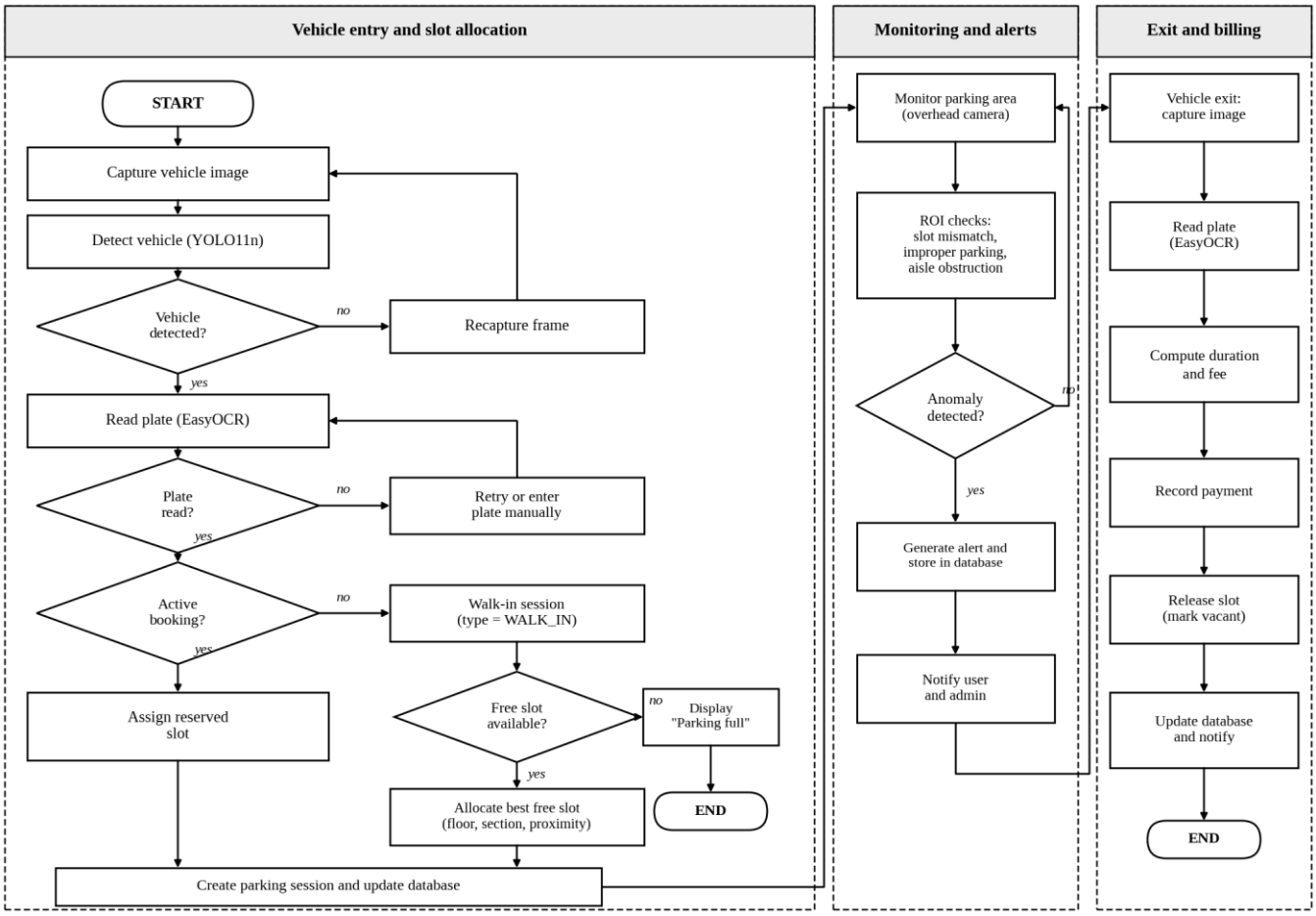


Fig. 2. Flowchart of the proposed parking-session workflow that includes vehicle entry, reservation check, walk-in process, dynamic slot allocation, anomaly detection, exit process, billing and slot release.

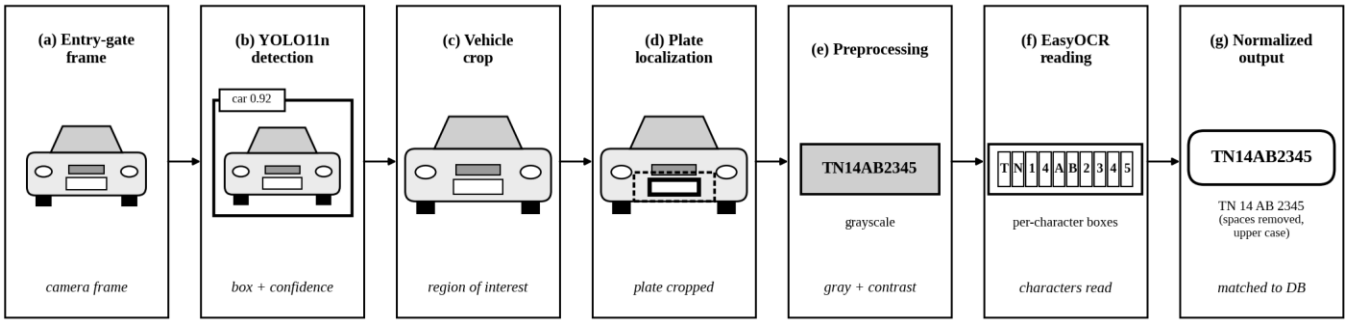


Fig. 3. Vehicle detection and license plate recognition pipeline: (a) input frame, (b) YOLO11n detection, (c) vehicle cropping, (d) plate localization, (e) pre-processing, (f) EasyOCR reading and (g) normalized output. Scheme of illustration.

E. Availability Prediction

With one output for each prediction horizon, a Random Forest regressor [13] predicts the quantity of available slots 30, 60, and 90 minutes in advance. The time of day, the day of the week, and the present occupancy are among its inputs. Since there is currently insufficient real parking-history data, the model is trained using artificial demonstration data. As a result, the produced numbers show that the prediction pipeline functions properly and shouldn't be taken as an indication of actual forecasting accuracy (Section VI-D).

F. Parking Anomaly Detection

An above camera keeps an eye on the parking lot following car entry. YOLO11n's vehicle bounding boxes are compared to pre-established regions of interest (ROIs) that correspond to driving aisles and parking spaces. Three forms of anomalies are taken into consideration: driving-aisle obstruction, inappropriate or duplicate parking, and slot mismatch. The trigger rules are listed in Table II and shown in Fig. 7. The overlap ratio for a slot ROI Rs and a detected vehicle bounding box B is defined as

$$\rho_s = \text{area}(B \cap R_s) / \text{area}(B) \quad (1)$$

When a vehicle is found in a slot ROI that differs from its designated slot, a slot-mismatch situation is created. When a vehicle extends beyond of its designated ROI or overlaps two or more slot ROIs over a configured threshold (i.e., ρs exceeds the configured ratio for two or more slots), it is considered improper or double parking. When a car stays inside a designated aisle ROI for longer than the configured time threshold Taisle, an aisle-obstruction alert is generated.

TABLE II
ANOMALY DETECTION RULES

Anomaly	Trigger
Slot mismatch	Vehicle detected in a slot ROI other than its assigned slot, e.g., assigned C04 but detected in C05.
Double / improper parking	Vehicle overlaps two or more slot ROIs above the configured ratio or extends outside its assigned ROI.
Aisle obstruction	Vehicle remains inside an aisle ROI longer than the configured threshold T_{aisle}.

V. IMPLEMENTATION

A. Database

The logical schema contains eight entities: users, vehicles, parking slots, bookings, parking sessions, anomalies, payments, and notifications (Fig. 4). Vehicles are associated with users, while bookings connect vehicles and slots with their reservation periods. Each parking session records the vehicle, assigned slot, entry time, exit time, duration, session type, and status. Anomaly records are linked to parking sessions, payments are generated from completed sessions, and notifications are associated with users.

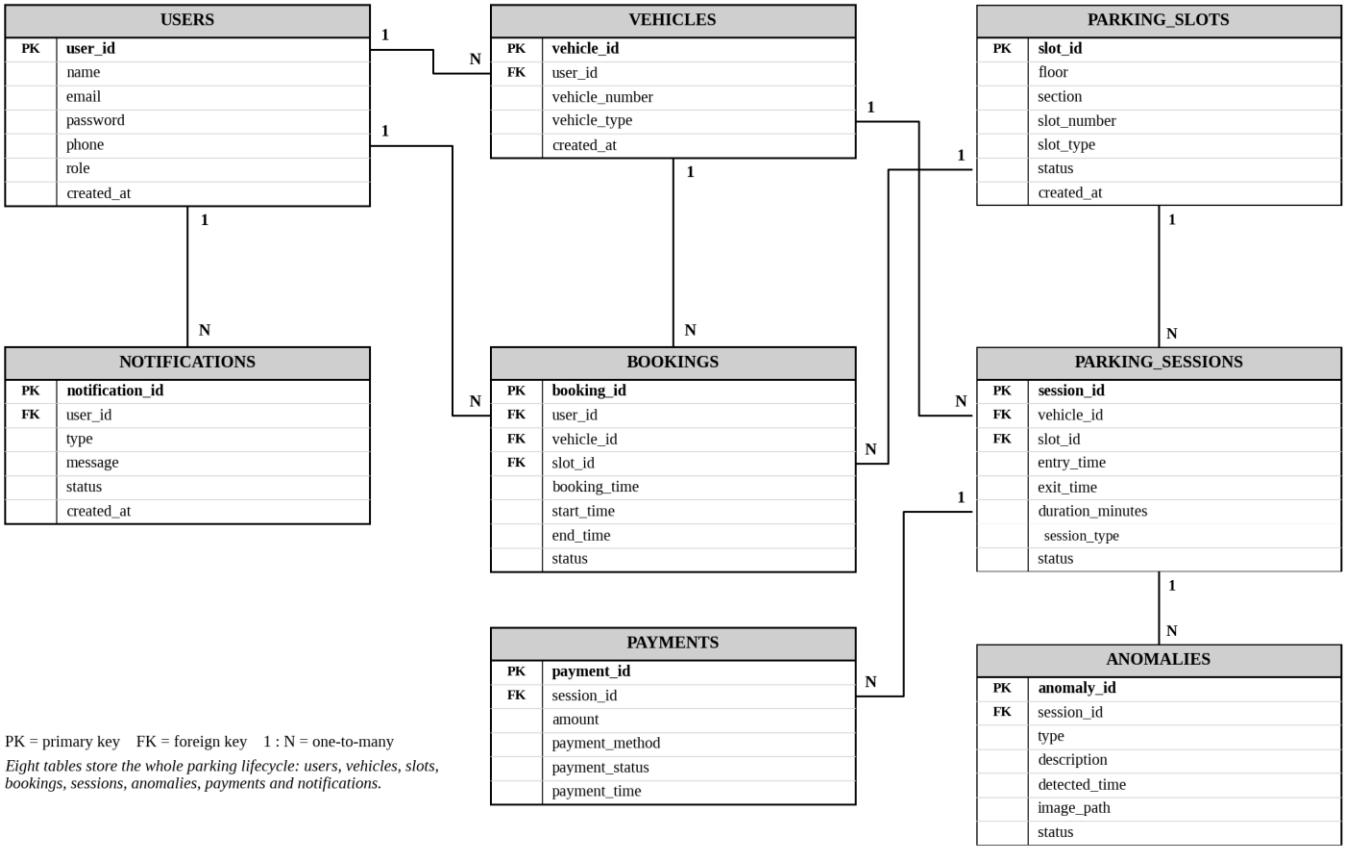


Fig. 4. Entity-relationship diagram of the SQLite database (PK = primary key, FK = foreign key).

G. Dynamic Billing

The billing module calculates the parking charge when the vehicle exits the facility. The total fee is calculated using configurable prototype rules based on the parking duration and applicable parking violations.

$$F_{total} = F_{base} + F_{duration} + F_{violation} \quad (2)$$

F_{base} represents the base parking charge, $F_{duration}$ represents the duration-based charge, and $F_{violation}$ represents any applicable charge associated with a detected parking violation. The billing rules are configurable prototype rules and do not represent real commercial parking tariffs.

B. Admin and User Portals

The Admin portal provides a centralized view of the parking facility. It displays total, available, occupied, and reserved slots, together with floor-level parking information. Administrators can monitor vehicle entry and exit, manage bookings, inspect individual slot details, review parking anomaly alerts, monitor walk-in vehicles, view parking history, manage billing records, and inspect availability predictions. Fig. 5 shows a wireframe of the main Admin views.

The User portal provides registration and login, slot booking, booking status, current parking-session information,

Find My Vehicle, parking history, notifications, and parking-bill information. Users receive notifications regarding booking confirmation, slot assignment, vehicle entry, anomaly alerts, and bill generation. The system is written in Python with FastAPI and Uvicorn, uses OpenCV [14] for image handling and SQLite for storage, and the portals are plain HTML, CSS and JavaScript (Table III).

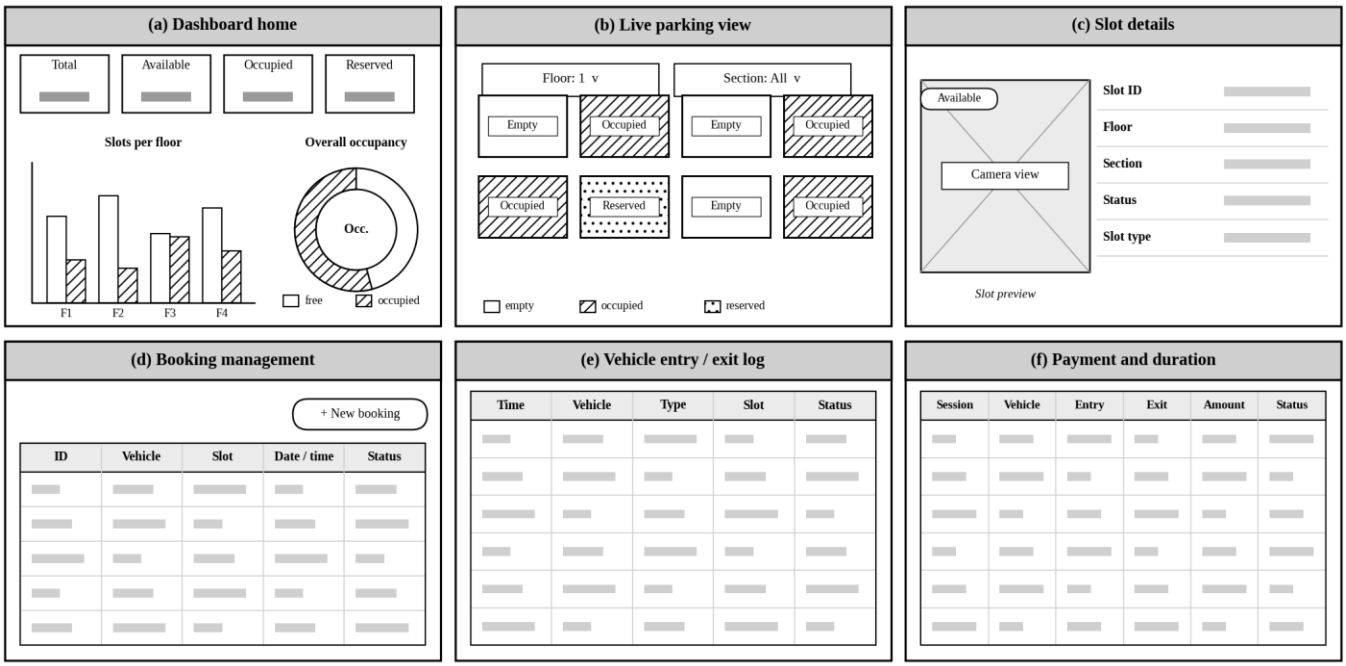


Fig. 5. Wireframe of the Admin dashboard: (a) dashboard home, (b) live parking view, (c) slot details, (d) booking management, (e) vehicle entry/exit log and (f) payment and duration.

VI. Experiments and Results

A. Experimental Setup

The prototype currently runs as a software system on a laptop or PC with a camera. Table III lists the software configuration. In the second phase, a physical sensing and barrier interface built around an ESP32-S3 microcontroller, HC-SR04 ultrasonic sensors, RGB LEDs and a barrier mechanism will be added.

TABLE III
Software Configuration

Component	Technology
Language	Python 3.12
Backend / server	FastAPI / Uvicorn
Database	SQLite
Vehicle detection	YOLO11n
OCR	EasyOCR
Computer vision	OpenCV
Prediction	Random Forest
Frontend	HTML / CSS / JavaScript
Hardware (current)	Laptop / PC + camera
Hardware (Phase 2)	ESP32-S3, HC-SR04, RGB LEDs, barrier

B. Functional Tests

Table IV summarizes the functional tests. A test is marked as passed only after it has been run on the prototype. The ten core workflow tests passed. The four anomaly and billing checks are still pending and are the immediate next step.

TABLE IV
Functional Test Status

Test	Expected	Status
Vehicle detection	Vehicle detected	Passed
Plate recognition	Plate extracted	Passed
Booking verification	Booking identified	Passed
Walk-in vehicle	Session created	Passed
Slot allocation	Vacant slot assigned	Passed
Vehicle entry	Session started	Passed
Find vehicle	Slot displayed	Passed
Vehicle exit	Session closed	Passed
Slot release	Slot becomes vacant	Passed
Prediction	Future availability generated	Passed
Slot mismatch	Alert raised	Pending
Double / improper parking	Alert raised	Pending
Aisle obstruction	Alert raised	Pending
Dynamic billing	Correct total fee	Pending

C. Results and Discussion

Vehicle entry. At the entry gate, the plate TN14AB2345 was recognized and the vehicle was assigned to a parking slot on Floor 2. *Walk-in handling.* A vehicle without an active booking was given a walk-in session and assigned an available slot instead of being rejected. *Find My Vehicle.* A search for TN58XY1234 returned Floor 1, Section A, Slot A5.

Prediction. In a sample run, the Random Forest model predicted 17, 14, and 13 free slots at 30, 60, and 90 minutes, respectively, against reference values of 18, 15, and 13 (Fig. 6). Since the model was trained using synthetic demonstration data,

these results demonstrate the operation of the prediction pipeline and do not establish real-world forecasting accuracy.

Anomaly detection. Fig. 7 illustrates the decision logic for slot mismatch, double or improper parking, and driving-aisle obstruction. For example, if a vehicle assigned to Floor 2, Slot B18 is detected in Floor 1, Slot A05, a slot-mismatch alert is generated. The automated anomaly checks remain pending quantitative validation on labelled parking events (Table IV).

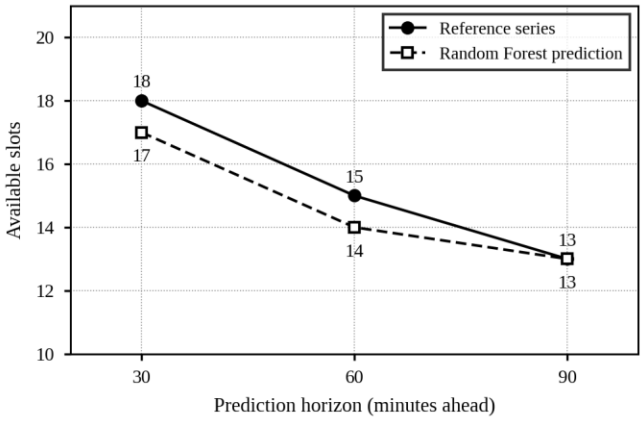


Fig. 6. Predicted and reference number of available slots at 30, 60 and 90 minutes (sample run, synthetic data).

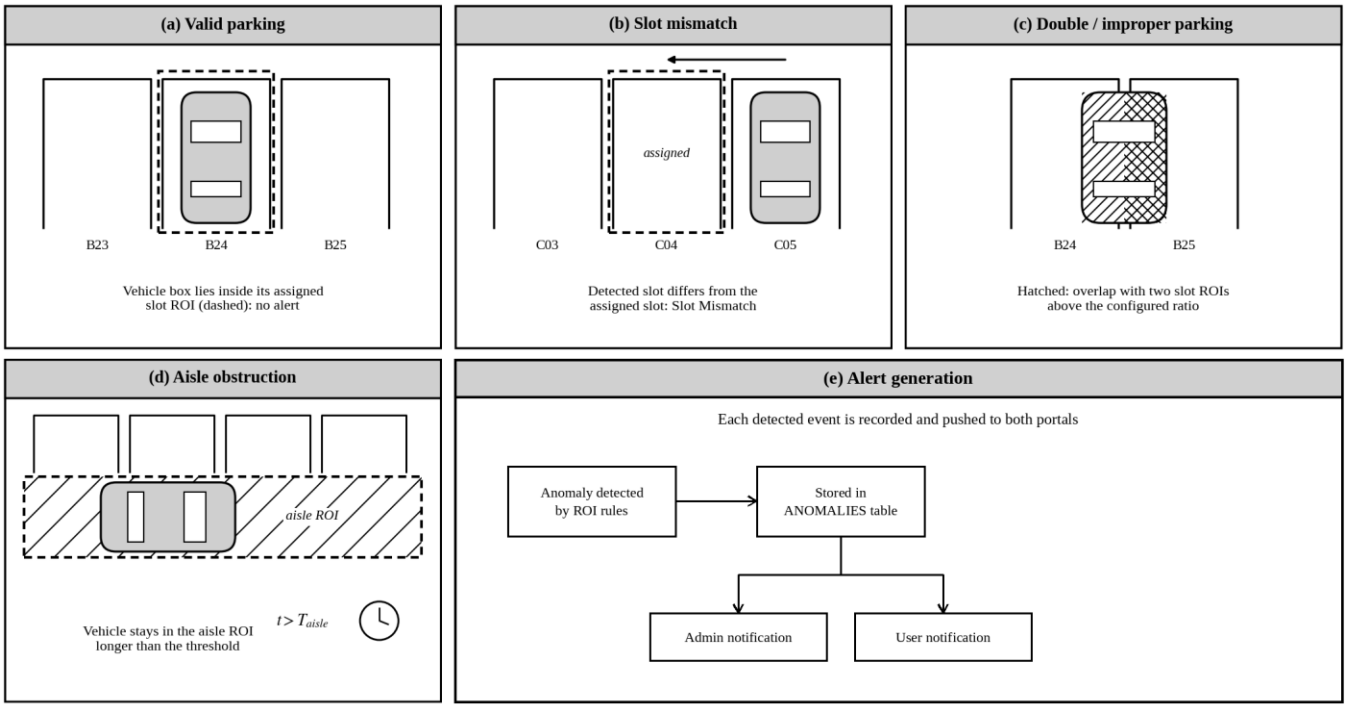


Fig. 7. Parking anomaly detection logic: (a) valid parking, (b) slot mismatch, (c) double or improper parking, (d) aisle obstruction and (e) alert generation. Schematic top views.

D. Evaluation Plan

The demonstration shows that the prototype follows a vehicle from entry to exit and treats unbooked vehicles as legitimate parking sessions. It is not a large-scale performance evaluation. Future quantitative evaluation will measure vehicle detection precision, recall and mAP on labelled gate frames, plate-level exact-match accuracy for ANPR, MAE and RMSE of availability prediction using real occupancy logs, precision and recall of each anomaly-alert category against manually labelled events, billing-rule correctness, and end-to-end processing latency.

VII. Limitations and Future Work

The current prototype has several limitations. OCR accuracy depends on plate visibility, resolution, lighting, and camera angle. Vehicle detection and ROI-based anomaly detection depend on camera placement and calibration. The prediction model is trained on synthetic demonstration data, while the billing rules are configurable prototype rules rather than real commercial tariffs. Vehicle detection, OCR, and anomaly detection have not yet been quantitatively validated using a large real-world labelled dataset. Physical parking sensors and barrier hardware are reserved for Phase 2. Number plates are personal data; a deployment should store only normalized plate strings, restrict access to them and follow the applicable data-protection rules, for example India's Digital Personal Data Protection Act, 2023.

Future work includes ESP32-based slot sensing using ultrasonic sensors, automatic barrier integration, an Android application with Firebase notifications, and cloud-based deployment. On the software side, future development will

focus on collecting a larger real-world parking dataset, improving vehicle tracking, developing learning-based parking anomaly detection, integrating online payment facilities, and evaluating the system under different camera positions, lighting conditions, and parking environments.

VIII. Conclusion

This paper presented an AI-driven smart parking management system covering the complete parking lifecycle from vehicle identification and reservation verification to walk-in handling, dynamic slot allocation, parking monitoring, billing, and slot release. The system integrates YOLO11n-based vehicle detection, EasyOCR-based ANPR, FastAPI, SQLite, Random Forest availability prediction, ROI-based parking anomaly detection, and separate Admin and User portals within a unified parking-session workflow.

The prototype demonstrates that booked and unbooked vehicles can be processed through the same parking management framework, with available slots dynamically assigned and vehicle locations stored for later retrieval. The anomaly module provides rule-based detection of slot mismatch, double or improper parking, and driving-aisle obstruction. The prediction module demonstrates future availability estimation using synthetic parking-history data, while the billing module provides configurable duration- and violation-based charging.

The current implementation is a Phase 1 software prototype and has not yet been quantitatively evaluated using a large real-world labelled dataset. Future work will therefore focus on real parking-data collection, anomaly-detection evaluation, physical sensor integration, online payment, mobile notifications, and deployment in larger parking environments.

References

[1] T. Lin, H. Rivano, and F. Le Mouël, "A survey of smart parking solutions," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 12, pp. 3229–3253, Dec. 2017.

[2] P. R. L. de Almeida, L. S. Oliveira, A. S. Britto Jr., E. J. Silva Jr., and A. L. Koerich, "PKLot—A robust dataset for parking lot classification," *Expert Syst. Appl.*, vol. 42, no. 11, pp. 4937–4949, 2015.

[3] G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, "Deep learning for decentralized parking lot occupancy detection," *Expert Syst. Appl.*, vol. 72, pp. 327–334, 2017.

[4] R. Laroca et al., "A robust real-time automatic license plate recognition based on the YOLO detector," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, 2018, pp. 1–10.

[5] S. M. Silva and C. R. Jung, "License plate detection and recognition in unconstrained scenarios," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 593–609.

[6] Y. Geng and C. G. Cassandras, "New 'smart parking' system based on resource allocation and reservations," *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 3, pp. 1129–1139, Sep. 2013.

[7] Y. Zheng, S. Rajasegarar, and C. Leckie, "Parking availability prediction for sensor-enabled car parks in smart cities," in *Proc. IEEE Int. Conf. Intell. Sensors, Sensor Netw. Inf. Process. (ISSNIP)*, 2015, pp. 1–6.

[8] E. I. Vlahogianni, K. Kepaptsoglou, V. Tsetsos, and M. G. Karlaftis, "A real-time parking prediction system for smart cities," *J. Intell. Transp. Syst.*, vol. 20, no. 2, pp. 192–204, 2016.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.

[10] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[11] JaidedAI, "EasyOCR," GitHub repository. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[12] A. Henaien, H. Ben Elhadj, and L. Chaari Fourati, "TinyML enabled smart parking dynamic slots computing and license plate recognition," in *Proc. Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, Ayia Napa, Cyprus, 2024, pp. 1583–1588, doi: 10.1109/IWCMC61514.2024.10592456.

[13] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[14] G. Bradski, "The OpenCV library," *Dr. Dobb's J. Softw. Tools*, 2000.